The specification is a strong foundation for a focused, vertical-first captioning tool, but it is under-specified for production: it omits the core editing/alignment controls, upload validation, render lifecycle, accessibility, security, observability, and many mobile/desktop interaction states. The biggest risks are silent auto-trimming, weak transcript correction, unreliable word timing, and an incomplete render/error/recovery contract. Groq supports word-level timestamps via `verbose_json` and `timestamp_granularities: ["word"]`, but word timestamps add latency, so the product needs a fallback and user-visible processing states.

Executive audit

Area	Assessment	Priority
Core captioning workflow	Good concept, but missing transcript editing, timing correction, speaker handling, and re-transcription controls	P0
Upload and duration policy	Silent trimming of long uploads is risky and likely to create user dissatisfaction	P0
Render pipeline	Missing job states, progress, cancellation, retries, idempotency, artifact lifecycle, and cost controls	P0
Mobile UX	Only a basic breakpoint and touch-target rule; needs a mobile-first layout and interaction model	P0

Desktop UX	Layout is described at a high level but lacks panels, state transitions, preview controls, and empty/loading/error states	P1
Accessibility	No keyboard, screen-reader, focus, reduced-motion, contrast, or caption-accessibility requirements	P0
QA	Tests are too narrow; needs E2E, visual, performance, accessibility, security, and failure-path coverage	P0
Technical architecture	Missing API contracts, storage lifecycle, queueing, observability, rate limiting, and data retention	P0

Critical product gaps

1. Do not silently truncate uploads

The spec says files longer than 90 seconds are accepted and the first 90 seconds are used. This is a major UX and trust issue: a creator may upload a 3-minute Reel, render it, and discover that the final two minutes were discarded.

Required change

- Detect media duration immediately after upload or client-side metadata probing.
- Before transcription, show a clear duration modal:
  - “Your video is 2:47. Auto Caption Generator creates 90-second vertical videos.”
  - Offer: **Use first 90 seconds**, **Choose 90-second segment**, or **Cancel / upload a shorter clip**.

- If the user selects “Use first 90 seconds,” retain this choice visibly in the studio header: `Using 0:00–1:30`.
- Do not begin transcription until the user confirms the selected segment.
- Preserve the original upload until render completion or a defined retention period, so the user can change the selected segment without re-uploading.

Why: Silent data loss is unacceptable in a creator tool. Explicit segment selection also enables a future “choose the best hook” workflow without changing the current 90-second constraint.

2. Add a real transcript editing and alignment workflow

The spec promises an “editorial transcript,” but provides no requirements for how users edit it, what happens to timings after edits, or how to fix poor word boundaries.

Required change

Implement a transcript editor with these capabilities:

- Display transcript as editable word chips or a text block with word-level timing metadata retained behind the scenes.
- Allow users to:
  - Correct spelling and proper nouns.
  - Delete filler words such as “um,” “uh,” and repeated words.
  - Merge incorrectly split words.
  - Split a word only when the underlying audio timing supports it.
  - Change capitalization and punctuation without destroying timing.
- Preserve word start/end timestamps when text is edited; if a word is deleted, remove its timing entry; if words are merged, use the earliest start and latest end.
- Provide **Re-transcribe** after meaningful edits if the user wants a fresh AI pass.
- Provide **Reset transcript** to restore the original transcription.
- Show a transcript confidence indicator or low-confidence word highlight where available.
- Add search-and-replace for recurring misspellings, brand names, and Hinglish terms.
- Persist the edited transcript locally during the session and server-side against the render job.

Implementation instruction for Antigravity: model the transcript as an immutable array of `CaptionToken` objects:

```ts
type CaptionToken = {
  id: string;
  text: string;
  startMs: number;
  endMs: number;
  confidence?: number;
  speaker?: string;
  edited?: boolean;
};
```

Do not store the transcript as a plain string only. Render timing must derive from token timing, not from string indexes.

3. Add caption grouping and line-break control

The spec describes word highlighting but not how words are grouped into readable caption lines. This directly affects readability, retention, and visual quality.

Required change

Add caption grouping controls:

- **Words per caption group:** 1–6, default 3.
- **Maximum characters per line:** enforce a responsive maximum, e.g. 18–24 characters for 9:16.
- **Line-break mode:** Auto, manual line break, or balanced.
- **Punctuation-aware grouping:** avoid breaking after mid-sentence punctuation where possible.
- **Manual caption-group editor:** allow users to move a word to the previous or next group.

- **Timing mode:** word-by-word highlight, phrase highlight, or block reveal.
- **Highlight style:** active word only, active phrase, or progressive fill.

Why: “Karaoke” captions fail when a group contains too many words, wraps awkwardly, or highlights words out of spoken order. This is a core quality control for a caption product.

AI and transcription improvements

4. Make transcription robust and transparent

The current fallback from Groq Whisper to Gemini 2.0 Flash is mentioned, but the fallback behavior is undefined.

Required change

- Define a transcription pipeline:
  - Validate and normalize media.
  - Extract mono 16 kHz audio.
  - Submit to Groq Whisper with `response_format: "verbose_json"` and word timestamps.
  - Validate returned word timing continuity.
  - If Groq fails, rate-limits, times out, or returns invalid timing data, retry with exponential backoff.
  - If retries fail, invoke Gemini fallback.
  - If both fail, show a recoverable error and do not charge credits.
- Show transparent provider status in the UI only when useful, e.g. “Using alternate transcription engine,” without exposing internal vendor details unnecessarily.
- Add a language control:
  - Auto-detect.
  - English.
  - Hindi.
  - Hinglish / code-mixed English-Hindi.
  - Other supported languages.
- Add a transcription quality review step before styling:
  - Show detected language.

    - Show speech coverage.
    - Warn if fewer than 3 words or less than 2 seconds of speech are detected.
- Do not claim “millisecond-accurate” unless the pipeline validates timestamps; instead use “word-level timing” and display timing precision honestly.

Groq explicitly supports word and segment timestamp granularities, and word-level timestamps increase latency; the UI must account for this rather than implying instant transcription.

5. Handle non-English, multilingual, and noisy audio

The specification focuses on Hinglish but does not define behavior for mixed languages, music-heavy audio, overlapping speakers, accents, or poor microphone quality.

Required change

- Add an **audio quality check** after upload:
    - Detect silence, clipping, very low volume, and excessive background noise.
    - Offer actionable guidance: “Move closer to the microphone,” “Reduce background music,” or “Try a clearer recording.”
- Add optional **speech enhancement** or **noise reduction** as an explicit pre-processing toggle, with a clear note that it may alter voice timbre.
- Detect multiple speakers where feasible and provide a speaker-label view in the transcript.
- For code-mixed content, allow a user-selected transcription language and a separate display-language preference.
- Add a “transcription quality” state: `excellent`, `good`, `review-recommended`, and `poor`.

Studio UX improvements

6. Redesign the desktop workspace

The current three-row layout is too abstract. A caption studio needs simultaneous access to media, transcript, style, and preview.

Required desktop layout

Use a three-column workspace at `>=1280px`:

Region	Contents
Left rail	Step navigation, project name, media summary, duration selector, save state
Center workspace	Step-specific editor: transcript editor, music controls, caption style controls, or render summary
Right preview	Persistent 9:16 preview, playback controls, timeline, safe-zone overlay, and render CTA

Required interactions

- Keep the 9:16 preview visible during Steps 2–4; users must see style changes immediately.
- Add playback controls: play/pause, scrub, jump to previous/next caption group, loop selected caption group, mute original audio, and music-volume control.
- Add a timeline with:
    - Playhead.
    - Caption-group blocks.
    - Speech segments.
    - Music waveform.
    - Selected segment handles.
- Add undo/redo for transcript and caption-style changes.
- Add autosave with a visible state: “All changes saved,” “Saving…,” or “Unsaved changes.”
- Add keyboard shortcuts:
    - Space: play/pause.

    - J/K/L: backward, pause, forward.
    - Arrow keys: seek.
    - Cmd/Ctrl + Z: undo.
    - Cmd/Ctrl + Shift + Z: redo.
    - Cmd/Ctrl + Enter: render.
- Prevent accidental navigation away with unsaved changes.

7. Improve mobile as a first-class experience

The current mobile requirement only collapses stepper tabs and sets 44px touch targets.

That is insufficient for a media-editing product.

Required mobile layout

Use a bottom-sheet and full-screen-panel model:

- **Top bar:** project name, save status, help, and close.
- **Step pill bar:** compact horizontal stepper with progress; tapping a completed step returns to it.
- **Main canvas:** one primary task per screen.
- **Bottom action bar:** primary action, secondary action, and progress indicator.
- **Preview behavior:**
    - On phones, show a persistent mini preview during Steps 2–3.
    - Tap the mini preview to expand it full-screen.
    - During Step 4, show the full-screen preview with sticky **Render** button.
- **Transcript editing:**
    - Use large tap targets for word chips.
    - Provide a bottom-sheet editor for selected words.
    - Support native mobile text selection, copy, paste, and undo.
- **Style selection:**
    - Use horizontally scrollable style cards with real animated previews.
    - Avoid tiny font-size dropdowns; use segmented controls or sliders.
- **Music audition:**
    - Do not autoplay audio.
    - Use explicit play/pause buttons.
    - Stop preview audio when another track starts.
- **Touch targets:** minimum 44 × 44 px; ensure adjacent controls have at least 8 px spacing.

- **Orientation:** optimize for portrait; support landscape without breaking the sticky action bar or hiding the preview.

Why: Mobile creators will often upload, review, and publish directly from a phone. The current spec treats mobile as a resized desktop rather than a distinct workflow.

8. Define all UI states

The specification lacks empty, loading, partial, error, and success states.

Required states

State	Required UI
Empty project	Explainer, supported formats, duration guidance, sample-project option
Uploading	Progress percentage, speed, cancel, warning not to close tab
Transcribing	Animated progress, elapsed time, cancel, explanation that long audio takes longer
Transcript ready	Transcript editor, confidence warning if applicable
Style preview	Live animated preview, selected style label, reset-style action

Render queued	Position in queue if applicable, estimated completion range
Rendering	Progress percentage, stage label, cancel where technically feasible
Render complete	Preview, download MP4/SRT/VTT, share/copy link, create another
Render failed	Plain-language reason, retry, contact support, confirmation that credits were restored
Offline	Persistent banner, disabled render action, local draft preservation

Do not use a single generic spinner for upload, transcription, styling, queueing, and rendering. Each stage needs a distinct label and expected-duration communication.

Visual design and accessibility

9. Strengthen accessibility

The current design tokens do not address accessibility sufficiently. WCAG requires captions for prerecorded audio and minimum contrast for text; large text requires at least 3:1 contrast, while normal text requires 4.5:1.

Required change

- Ensure all UI text meets WCAG AA contrast.

- Do not rely on color alone to indicate active caption words; combine color with weight, background pill, underline, scale, or outline.
- Provide a **high-contrast caption mode**.
- Ensure subtitle styling remains readable over bright, dark, and busy video backgrounds:
    - Optional stroke/outline.
    - Optional shadow.
    - Optional semi-opaque backdrop.
    - Automatic contrast adjustment based on sampled background luminance.
- Ensure focus rings are visible on all interactive controls.
- Make the entire studio operable by keyboard.
- Add `aria-live` announcements for:
    - Upload completion.
    - Transcription completion.
    - Render progress milestones.
    - Errors and credit restoration.
- Respect `prefers-reduced-motion` by reducing or disabling nonessential caption animation in the editor preview; preserve the user-selected rendered animation only when explicitly chosen.
- Provide captions for product demo videos and accessible labels for all icon-only buttons.

10. Improve safe zones and device coverage

The current rule specifies a 100 px bottom and 80 px right margin, but social-platform UI varies and the left margin is unspecified.

Required change

- Replace fixed pixel margins with a safe-zone overlay system:
    - TikTok safe zone.
    - Instagram Reels safe zone.
    - YouTube Shorts safe zone.
    - Custom safe zone.
- Show a visual safe-area overlay in the preview.
- Add top safe-zone protection for usernames, captions, and platform headers.
- Add a warning when text intersects a selected platform's unsafe area.
- Allow users to choose platform preset before final render.

- Keep 1080 × 1920 as the render target, but define all positioning in normalized coordinates or Remotion-relative units so future resolutions do not break layout.

Technical architecture gaps

11. Define the complete render lifecycle

"AWS Lambda cloud rendering" is not an implementation-ready specification. Remotion Lambda splits renders into concurrent chunks; concurrency, `framesPerLambda`, timeout, memory, cost, and failure recovery all need explicit decisions.

Required change

Define the render job state machine:

```
text

DRAFT → UPLOADING → UPLOADED → TRANSCRIBING → TRANSCRIPT_READY
→ STYLING → PRE_RENDER_VALIDATION → QUEUED

→ RENDERING → COMPLETED | FAILED | CANCELED | EXPIRED
```

For each state, specify:

- Allowed user actions.
- Server-side persistence.
- UI state.
- Retry policy.
- Credit/quota behavior.
- Error codes.
- Telemetry events.

Implementation instructions for Antigravity:

- Use an idempotent render-request ID tied to the user, project, and input hash.

- Store render jobs in a durable queue/database record, not only in client state.
- Use signed, short-lived upload URLs; do not expose S3 credentials to the browser.
- Validate MIME type, extension, file size, duration, codec, and audio presence before accepting a job.
- Run server-side media probing; client-side probing is a UX aid, not a security boundary.
- Store original media, extracted audio, transcript JSON, render settings, output MP4, SRT, VTT, thumbnails, and render logs with separate retention policies.
- Use signed, expiring download URLs for outputs.
- Implement a render-progress endpoint using polling or WebSocket/SSE, with a polling fallback.
- Implement cancellation semantics:
    - Before render starts: cancel immediately.
    - During Remotion render: mark canceled, attempt to stop the render, and reconcile credits asynchronously.
    - After completion: do not allow cancellation.
- Implement retry limits and exponential backoff for transient provider errors.
- Preserve partial outputs only when safe; otherwise clean up orphaned artifacts.

Remotion recommends tuning Lambda memory and concurrency rather than defaulting to high memory, and warns that excessive concurrency can create overhead or instability.

12. Add scalability, cost, and abuse controls

The free tier offers five daily videos with no credit cost, but the spec does not define abuse prevention or infrastructure limits.

Required change

- Add per-user and per-IP rate limits for:
    - Uploads.
    - Transcription requests.
    - Render submissions.
    - Download requests.
- Add daily, monthly, and concurrent-job limits.
- Queue render jobs rather than allowing unlimited parallel Lambda renders.

- Add priority tiers if paid users should render faster.
- Track cost per job: transcription provider usage, S3 storage, Lambda duration, and output storage.
- Add anomaly detection for duplicate uploads, repeated failures, and abnormal render volume.
- Add an admin operations view for failed jobs, stuck jobs, queue depth, provider errors, and credit-restoration failures.
- Define retention:
    - Source media: e.g. 7 days.
    - Rendered output: e.g. 30 days.
    - Transcript: user-controlled deletion.
    - Delete artifacts upon account deletion request.

13. Secure user media and transcripts

The spec does not address privacy, consent, moderation, or data handling.

Required change

- State that uploaded media and transcripts may be processed by third-party AI providers.
- Obtain explicit consent where legally required.
- Provide a data-processing disclosure before first upload.
- Do not send unnecessary metadata to transcription providers.
- Encrypt media at rest and in transit.
- Use least-privilege IAM roles.
- Scan uploaded files for malware where appropriate.
- Block or flag prohibited content according to platform policy.
- Provide export and deletion controls for user projects.
- Redact or restrict logs so URLs, user IDs, and media identifiers are not exposed unnecessarily.

Error handling and QA

14. Expand error taxonomy

The current error handling covers only silent audio, corrupt files, and network interruption. This is insufficient.

Required error catalogue

Error	User message	Recovery
Unsupported codec	“This file format isn’t supported. Try MP4, MOV, or WebM.”	Re-upload
File too large	Show maximum size and compression guidance	Re-upload
No audio track	“We couldn’t find an audio track.”	Upload a file with audio
No speech detected	Existing behavior, but allow user to override if music/ambient audio is intentional	Retry or cancel
Transcription timeout	“Transcription is taking longer than expected.”	Retry, fallback, cancel
Transcription provider outage	“Transcription is temporarily unavailable.”	Retry later

Invalid word timings	“We couldn’t align captions reliably.”	Re-transcribe or manual timing
Render queue full	“Render queue is busy. Your place is reserved.”	Wait or cancel
Render timeout	“Rendering took too long.”	Retry
Lambda concurrency failure	Retry with reduced concurrency	Automatic retry
Output generation failure	“We couldn’t generate the final video.”	Retry, restore credits
SRT/VTT export failure	“Subtitle download failed.”	Retry download
Credit restoration failure	“Your render failed. We’re restoring your credits.”	Support ticket if unresolved

Every error needs a stable machine-readable code, user-safe message, retryability flag, telemetry event, and support-context ID.

15. Expand the QA plan

`npx tsc --noEmit`, a frame-sync test, and a safe-zone check are useful but not enough for production.

Required test coverage

- **Unit tests**
  - Transcript token editing.
  - Timing preservation after edits.
  - Caption grouping.
  - Safe-zone calculations.
  - SRT/VTT generation.
  - Duration trimming.
  - Credit reservation and restoration.
- **Integration tests**
  - Upload → probe → transcribe → edit → render request.
  - Groq failure → retry → Gemini fallback.
  - Render success, failure, timeout, cancellation, and retry.
  - Concurrent render limits.
- **E2E tests**
  - Desktop and mobile user journeys.
  - Upload from iOS Safari, Android Chrome, and desktop browsers.
  - Keyboard-only flow.
  - Screen-reader announcements.
- **Visual regression tests**
  - All 32 caption themes.
  - Small/medium/large font sizes.
  - Bottom/center/top alignment.
  - Long words, emojis, punctuation, mixed scripts, and very short speech.
- **Performance tests**
  - Upload progress on slow 3G-like networks.
  - Preview responsiveness with 90-second media.
  - Render p50/p95 duration.
  - Memory and Lambda concurrency behavior.
- **Accessibility tests**
  - axe/WAVE automated scans.
  - Manual screen-reader testing.
  - Keyboard traversal.
  - Reduced-motion behavior.
- **Security tests**

- Unauthorized project access.
- Signed URL expiry.
- Malicious file upload.
- Rate-limit enforcement.
- Cross-user data isolation.

Implementation instructions for Antigravity

Implement the following in priority order, without changing the existing 9:16, 30 FPS, 90-second, Remotion-based product direction.

Po — Must implement

1. **Explicit 90-second segment selection**
   - Add duration detection before transcription.
   - Show a selection modal for media longer than 90 seconds.
   - Persist `selectedStartMs` and `selectedEndMs`.
   - Display the active range throughout the studio.
   - Never trim silently.
2. **Transcript token model and editor**
   - Replace plain-text transcript handling with `CaptionToken[]`.
   - Support edit, delete, merge, split, undo, redo, reset, and re-transcribe.
   - Preserve timing metadata through edits.
   - Autosave transcript state.
3. **Caption grouping controls**
   - Add words-per-group, maximum line length, line-break mode, highlight mode, and manual group movement.
   - Render from caption groups, not raw word arrays.
4. **Complete render lifecycle**
   - Implement `DRAFT → UPLOADING → UPLOADED → TRANSCRIBING → TRANSCRIPT_READY → STYLING → QUEUED → RENDERING → COMPLETED | FAILED | CANCELED | EXPIRED`.
   - Add progress, cancellation, retry, idempotency, signed URLs, cleanup, and credit reconciliation.
5. **Mobile-first interaction model**
   - Implement a persistent mini preview, bottom action bar, full-screen preview, bottom-sheet editors, and mobile-friendly transcript controls.
   - Ensure all primary actions meet 44 × 44 px minimum touch targets.
6. **Accessibility baseline**

- Add keyboard navigation, visible focus, ARIA labels/live regions, reduced-motion support, high-contrast mode, and contrast-safe caption rendering.

P1 — Should implement

7. **Safe-zone presets**
   - Add TikTok, Instagram Reels, YouTube Shorts, and custom safe-zone overlays.
   - Warn before rendering when captions intersect unsafe regions.
8. **Audio quality diagnostics**
   - Detect silence, clipping, low volume, and noise.
   - Give actionable remediation guidance before transcription.
9. **Language and speaker support**
   - Add language selection, Hinglish-friendly editing, speaker labels where available, and transcription-quality indicators.
10. **Desktop studio layout**
    - Implement the three-column workspace with persistent preview, timeline, playback controls, undo/redo, and autosave.
11. **Operational controls**
    - Add rate limits, queueing, concurrency limits, cost telemetry, admin observability, and retention policies.

P2 — Valuable enhancements

12. **Template marketplace**
    - Make the 32 themes data-driven, versioned, and previewable without a full render.
13. **Brand kit**
    - Allow saved fonts, colors, logo placement, and default caption settings.
14. **Batch mode**
    - Allow multiple short clips to be captioned with the same style.
15. **AI assist**
    - Add punctuation restoration, filler-word removal, hook/title suggestions, and translation or bilingual captions as opt-in features.

Acceptance criteria

The implementation should be considered complete when:

- A user uploading a video longer than 90 seconds must explicitly choose or confirm the 90-second segment before transcription.
- Transcript edits preserve word timing or provide a clear re-alignment path.
- Users can change caption grouping, line breaks, highlight mode, and safe-zone preset before render.
- The preview accurately reflects final caption position, font, animation, and safe-zone constraints.
- Render progress, failure, retry, cancellation, and credit restoration are visible and reliable.
- The studio is fully usable on a 375 × 667 viewport without horizontal scrolling or inaccessible controls.
- Keyboard-only users can complete the entire workflow.
- All automated tests, visual regression tests, accessibility checks, and failure-path tests pass.
- No user is charged for a failed render, and failed-render credit restoration is observable in logs and admin tooling.

The most important architectural principle is this: **the transcript, timing data, caption grouping, render settings, and render job must be first-class persisted entities—not transient UI state.** That change will make editing reliable, rendering recoverable, mobile UX coherent, and future features such as translation, speaker labels, templates, and batch processing practical. Groq's word-timestamp capability should be treated as an input to a resilient timing pipeline, not as a guaranteed final caption-timing solution.